

Master's Thesis
Final-Year Project

ACADEMIC YEAR 2025–2026

Transformer Compression

Using Genetic Algorithms

[Author Name] Candidate

[Supervisor Name] Supervisor

[University / Department]

$$\lambda_1 \geq \lambda_2 \geq \dots$$

$$\theta_{t+1} = \theta_t - \eta \nabla \mathcal{L}$$

$$p_{\theta}(y | x)$$

$$a_i^{(1)} = \sigma\left(\sum_{j=1}^n w_{i,j} a_j^{(0)} + b_i^{(0)}\right)$$

$$a^{(1)} = \sigma\left(W^{(0)} a^{(0)} + b^{(0)}\right)$$

Structure of the presentation

- 01 — **Foundations** Neural networks, the Transformer, and search
- 02 — **The compression problem** Compression families, prior work, and its limitation
- 03 — **Our method** Budget-constrained evolutionary search over architectures
- 04 — **Results and discussion** Encoder classification and decoder language modelling

Capable models, costly deployment

The Transformer is the dominant architecture for language and sequence modelling. The same scale that gives these models their capability also makes them expensive to deploy, in memory, latency, and energy, and often impractical on resource-limited hardware.



Model compression narrows the gap between training-scale capability and deployment-scale constraints. To compress a model in a principled way, we first need to know what it is built from, and in what units its cost is measured.

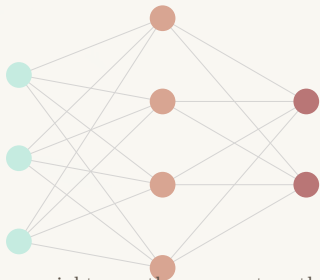
The remainder of Part I builds exactly this vocabulary: the network, the Transformer, the cost metrics, and the search method.

The neuron and the multilayer perceptron

A neuron forms a weighted sum of its inputs, adds a bias, and applies a nonlinearity:

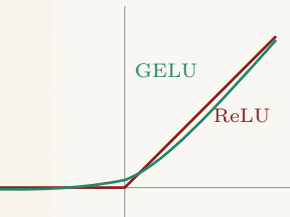
$$y = \phi(\mathbf{w}^\top \mathbf{x} + b)$$

Stacking neurons into successive layers yields a multilayer perceptron, whose depth and width set the capacity of the model.



These weights are the parameters that compression later removes, factorises, or stores at lower precision.

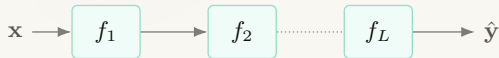
Nonlinearity and activation functions



Without a nonlinearity, a deep stack of layers collapses to a single linear map, regardless of depth. Activation functions such as ReLU and GELU give the network the expressive power to represent nonlinear relationships, while the softmax turns final scores into a probability distribution.

In a Transformer, most parameters sit in the feed-forward layers built around this nonlinearity.

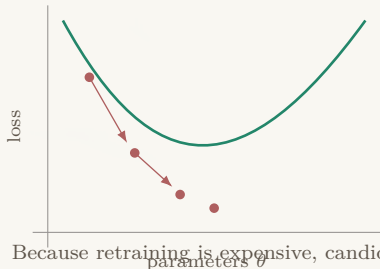
The forward pass and the loss



A network is a composition of layer functions that maps an input $\mathbf{x}$ to a prediction $\hat{\mathbf{y}}$. A loss function $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$ measures the discrepancy between the prediction and the target, and so defines the objective that training minimises.

The same loss is later used to judge whether a compressed model still behaves like the original.

Backpropagation and optimisation



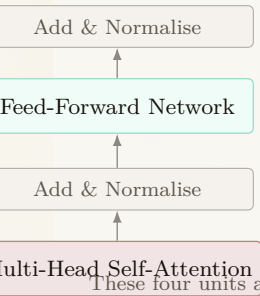
Because retraining is expensive, candidate compressed models are ranked by a cheap surrogate instead of being trained in full.

Backpropagation applies the chain rule to obtain the gradient of the loss with respect to every parameter. Gradient descent then updates the parameters in the direction that reduces the loss:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}$$

Repeated over a large dataset, this training step is the dominant computational cost of building a model.

The Transformer layer



A Transformer layer combines multi-head self-attention with a feed-forward network, linked by residual connections and normalisation. Layers stack to form the encoder and decoder.

- Attention heads
- Feed-forward neurons
- Whole layers
- Weight precision and rank

These four units are the knobs that every compression family in this work turns.

Measuring the cost of a model

The benefit of compression is measured in deployment cost. The search treats one of these quantities as a fixed budget and asks for the most accurate model that fits within it.

Parameters

Memory footprint

Arithmetic (MACs)

Bandwidth

Latency

Throughput

This budget becomes the binding constraint of the search: accuracy is maximised subject to it.

Genetic and multi-objective optimisation

A genetic algorithm maintains a population of candidate solutions and improves it through selection, crossover, and mutation. Multi-objective variants balance competing goals along a Pareto front. The approach suits discrete design variables for which no gradient is available.

